“AI Lost Item Finder Using Knowledge Representation and Backward Chaining”

G S PRANAV
COMPUTER SCIENCE AND ENGINEERING
RAJALAKSHMI ENGINEERING COLLEGE
CHENNAI, INDIA
pranav.gs.2024.cse@rajalakshmi.edu.in

M SENGATHIR
COMPUTER SCIENCE AND ENGINEERING
RAJALAKSHMI ENGINEERING COLLEGE
CHENNAI, INDIA
sengathir.m.2024.cse@rajalakshmi.edu.in

***Abstract*—*People frequently misplace everyday belongings such as identity cards, phones, wallets, keys, chargers, earphones, and books, and usually search by recalling recent activities and checking places at random. This paper proposes the AI Lost Item Finder, a lightweight knowledge-based system that applies classical Artificial Intelligence techniques, namely Knowledge Representation, First-Order Logic (FOL), and Backward Chaining, to reason about where a misplaced object is most likely to be. Relationships between items, containers, activities, and locations are stored as FOL-style facts and IF–THEN rules with certainty factors. Starting from the goal LikelyAt(Item, ?Location), the inference engine works backward through the rules, asks the user only the questions needed to prove or reject each hypothesis, and reports the most probable location, its confidence, the rule applied, alternatives, and a full reasoning trace. Implemented in Python 3 using only the standard library and tested on six scripted scenarios covering all six rules, the system returned the expected location in every case, asked 28 questions against 66 for a fixed questionnaire (about 58% fewer), and completed each inference in under one millisecond. The results are limited to scripted scenarios and do not represent a user study.*

Keywords*—*Lost Item Finder, Knowledge Representation, First-Order Logic, Backward Chaining, Certainty Factors, Expert Systems, Explainable AI.

I. Introduction

Misplacing personal belongings is one of the most common everyday problems. Students in particular move between home, classrooms, libraries, and canteens carrying small items such as identity cards, phones, earphones, chargers, and books. When one goes missing, the usual approach is to retrace activities from memory and search likely places one by one, which is slow, depends heavily on recall, and often repeats the same checks.

This work addresses the problem by proposing a knowledge-based system that reasons about where an item is likely to be. Instead of asking the user to remember everything at once, the system asks a short sequence of focused questions chosen by the reasoning process itself and explains why it recommends a particular location.

A. Need for the System

Manual searching is memory-dependent and unsystematic; tracking devices such as Bluetooth tags work only for items tagged in advance; and neither explains why one place is more likely than another. A system that represents knowledge explicitly, asks only relevant questions, and justifies its answer makes the search faster and easier to trust.

B. Aim and Objectives

The aim is to develop an intelligent, explainable system that identifies the most probable location of a misplaced personal item using information about the user’s recent activities. The objectives are to: represent items, containers, activities, and locations as FOL-style facts; encode domain knowledge as IF–THEN rules with certainty factors; implement a backward-chaining inference engine that works from location hypotheses to supporting facts; ask only the questions required to prove the current hypothesis and never repeat one; and report the most probable location, confidence,

rule applied, alternatives, and a step-by-step reasoning trace.

C. Scope

The project is a console-based Python system covering seven common items (college ID card, mobile phone, wallet, keys, charger, earphones, and book), ten locations, and six inference rules. It includes question generation, fact generation, backward chaining, certainty-factor ranking, and explanation. It uses no physical tracking hardware such as GPS, Bluetooth tags, or cameras and does not learn from previous sessions; these are left as future extensions.

D. Design Characteristics

The design emphasizes explicit knowledge representation rather than opaque models; goal-directed questioning that minimizes user effort; separation of the knowledge base from the inference engine so that new items, locations, and rules can be added as data; certainty-factor ranking that handles uncertain answers; and transparent output through a complete reasoning trace.

E. Organization of the Paper

Section II reviews the literature, Section III presents the architecture, Section IV describes the modules and the backward-chaining procedure, Section V covers implementation and results, Section VI discusses design rationale and practical considerations, and Section VII concludes with future work.

II. Literature Survey

The literature identifies several established techniques that form the conceptual basis of the proposed system (Table I).

Area	**Approach**
Expert systems	Knowledge base, inference engine, working memory, and explanation facility [4], [5]
Rule representation	IF–THEN production rules and Horn clauses in First-Order Logic [1], [6]
Chaining	Forward (data-driven) and backward (goal-driven) inference [1]
Unification	Variable binding that makes two logical expressions identical [1], [7]
Certainty factors	Numeric measure of rule strength, introduced in MYCIN [2], [3]

Table I: Approaches in the Literature

Knowledge-Based Expert Systems: A knowledge-based system separates domain knowledge from the mechanism that reasons over it. A typical system contains a knowledge base of facts and rules, an inference engine, a working memory holding facts about the current case, and an explanation facility [4], [5]. Such systems have been used for medical diagnosis, fault-finding, and decision support.

Rules and First-Order Logic: Production rules state that if a set of conditions holds, a conclusion may be drawn. Written in FOL, rules contain variables, so one rule applies to many cases. Rules with a single conclusion and a conjunction of conditions are Horn clauses, which permit efficient forward and backward chaining and underpin logic programming languages such as Prolog [6], [7].

Forward and Backward Chaining: Forward chaining starts from known facts and applies every satisfied rule until no new facts follow. Backward chaining starts from a goal, finds rules whose conclusion matches it, and treats their conditions as sub-goals proved from existing facts, other rules, or by asking the user [1]. Because every question costs the user effort, backward chaining suits this problem.

Certainty Factors: MYCIN introduced certainty factors to express how strongly a rule's conditions support its conclusion on a scale where 1.0 is full confidence [2], [3]. Here they rank candidate locations and decide when to stop asking questions.

Research Gap: Rule-based systems are widely applied to specialized domains, whereas existing lost-item solutions mostly rely on physical tags that must be attached in advance. The proposed system applies symbolic reasoning to locating untagged belongings using only the user's memory of recent activities, and makes the reasoning visible.

III. System Architecture

The system is a knowledge-based decision-support tool that receives the lost item and the user's answers and returns a ranked location with its explanation. It consists of a user interface, an item and question bank, a knowledge base, a session

fact store, a fact-generation component, an inference engine with rule matching, and an explanation and result module (Fig. 1, Table II).



Fig. 1: System architecture of the AI Lost Item Finder

A. Component Description

The user interface lets the user select the lost item and answer questions with Yes, No, Not sure, or a location. The item and question bank lists supported items, their properties, and natural-language question templates. The knowledge base stores the rules. Each answer is converted by the fact-generation component into a fact held in the session fact store. The inference engine proves the goal against the rules and facts, and the explanation module presents the result and trace.

Stage	Main responsibility
Input	Lost item selection; Yes / No / Not sure / location answers
Question bank	Item properties and question templates
Knowledge base	Six IF–THEN rules with certainty factors
Fact store	Static facts plus facts generated from answers
Inference	Backward chaining with unification
Output	Location, confidence, rule, alternatives, trace

Table II: System Processing Stages

B. Existing System and Limitations

In the existing approach a person relies on memory and manual searching. This is memory-dependent, lacks systematic reasoning, is time-consuming, has no formal representation of relationships between objects, activities, and locations, and provides no explanation. Tracking devices help only for items tagged in advance.

C. Knowledge Representation

Knowledge is expressed as FOL-style predicates. Static facts, such as Chargeable(Phone), PocketItem(Wallet), and TypicalPlace(Keys, FrontDoor), are loaded from the item bank. Askable predicates are obtained from the user, producing facts such as Removed(ID, Bag), RemovedAt(ID, StudyTable), KeptIn(Book, Bag), and ContainerAt(Bag, Classroom). Table III lists the rules.

Rule	IF (conditions)	THEN	CF
R1	LastUsedAt(?i, ?l) ∧ NotMovedSince(?i)	LikelyAt(?i, ?l)	0.90
R2	KeptIn(?i, ?c) ∧ ContainerAt(?c, ?l)	LikelyAt(?i, ?l)	0.85
R3	Removed(?i, ?c) ∧ RemovedAt(?i, ?l)	LikelyAt(?i, ?l)	0.80
R4	Chargeable(?i) ∧ Charging(?i) ∧ ChargedAt(?i, ?l)	LikelyAt(?i, ?l)	0.80
R5	PocketItem(?i) ∧ ChangedClothes	LikelyAt(?i, LaundryBasket)	0.70
R6	TypicalPlace(?i, ?l)	LikelyAt(?i, ?l)	0.30

Table III: Rules in the Knowledge Base

IV. Modules

A. Knowledge Base and Fact Generation Module

This module holds all the knowledge the system reasons with, in three parts. The item and question bank lists each item's properties (possible containers, whether it is chargeable, and where it is typically kept) and question templates, so that a sub-goal such as Removed(ID, Bag) is shown as "Did you take the college id card out of your bag?". The knowledge base stores the six rules, each as a tuple of identifier, condition predicates, conclusion, certainty factor, and an English description used in explanations; because rules are data, they can be extended without changing the inference code.

The session fact store starts with static facts and grows as the user answers. A Yes answer adds the ground fact; a location answer binds the variable in the question and adds the resulting fact; No and Not sure add nothing. All answers are cached so the same question is never asked twice.

B. Inference Engine and Explanation Module

The engine receives the goal LikelyAt(Item, ?l) and tries rules in descending order of certainty factor, so the most reliable explanations are tested first. Unification binds each rule's variables to the selected item, and the conditions are proved one by one, first against the fact store and, if the predicate is askable, by asking the user. Proof is depth-first using Python generators, so alternative bindings are explored when a condition fails. Each proved location is recorded with the certainty factor of the rule that produced it, keeping the highest value if several rules reach it.

The explanation module reports the most probable location, its confidence, the rule that produced it, alternative locations, the number of questions asked, and a reasoning trace of every goal, rule, question, and conclusion.

C. Backward-Chaining Procedure

A goal is a hypothesis about the item's location. The algorithm proceeds as follows:

1) Select the lost item and set the goal LikelyAt(Item, ?l).
2) Pick the next rule, in order of certainty factor, whose conclusion unifies with the goal.
3) Treat each condition of the rule as a sub-goal.
4) Prove each sub-goal from the fact store or, if askable, by asking the user and storing the answer as a fact.
5) If every sub-goal is proved, record the location with the rule's certainty factor.
6) Stop at the first conclusion with confidence of 0.75 or more; otherwise try the next rule.
7) If no strong rule succeeds, apply fallback rule R6 and report a low-confidence suggestion.

Within the architecture, backward chaining sits in the inference engine. It does not store knowledge or generate questions; it uses the rules and facts supplied by the knowledge-base module.

V. Implementation and Results

A. Experimental Setup

The system was implemented in Python 3 using only the standard library, so it runs on any machine with Python 3.8 or later. Tests were run with Python 3.13. The program comprises the item and question bank, knowledge base, inference engine, and explanation module in one file, together with a test script that runs scenarios with scripted answers. The knowledge base contains seven items, ten locations, six rules, and nine askable predicates; the stopping threshold is 0.75.

Each scenario was evaluated on whether the predicted location matched the expected one, the confidence, the rule used, the number of questions asked, and the inference time. To measure the benefit of goal-directed questioning, the questions asked were compared with a fixed questionnaire that asks one question for every askable condition of every applicable rule for that item. Six scenarios exercise every rule at least once, including one in which the user answers "Not sure" to everything (Table IV).

Element	Description
Language	Python 3 (standard library only)
Knowledge	7 items, 10 locations, 6 rules, 9 askable predicates
Stop threshold	Confidence ≥ 0.75
Test cases	Six scripted scenarios, S1–S6
Metrics	Correctness, CF, rule, questions, time

Table IV: Experimental Setup

B. Results

In scenario S1 (lost ID card), R1 fails because the user is not sure where the card was last used, and R2 fails for both bag and wallet. R3 succeeds when the user confirms the card was taken out of the bag at the study table, concluding LikelyAt(ID, StudyTable) with 80% confidence after five questions (Fig. 2).



Fig. 2: Reasoning trace and result for the lost college ID card (S1)

In S5 (book), the user says the book was moved, so R1 is rejected; R2 establishes that the book is in

the bag and the bag is in the classroom, giving Classroom at 85% after four questions. In S4 (wallet), R1–R3 fail, R4 is skipped because a wallet is not chargeable, and R5 gives LaundryBasket at 70%; being below the threshold, the fallback adds the bag and bedroom as alternatives at 30%. In S6 (earphones), the user is unsure of everything, so R6 suggests the usual places at a clearly stated 30%. Table V summarizes all scenarios.

ID	Item	Location	CF	Rule	Asked	Fixed
S1	ID card	StudyTable	80%	R3	5	13
S2	Phone	Bedroom	80%	R4	6	11
S3	Keys	FrontDoor	90%	R1	2	11
S4	Wallet	LaundryBasket	70%	R5	5	11
S5	Book	Classroom	85%	R2	4	9
S6	Earphones	Bag	30%	R6	6	11
	Total	6/6 correct			28	66

Table V: Results for All Test Scenarios

The predicted location matched the expected location in all six scenarios, and every rule R1–R6 produced the final answer at least once. Backward chaining asked 28 questions against 66 for a fixed questionnaire, about 58% fewer (Fig. 3). The saving is largest when a high-confidence rule succeeds early (S3, two questions) and smallest when the user is unsure and every rule must be tried (S6). Every inference completed in under one millisecond (maximum 0.11 ms).

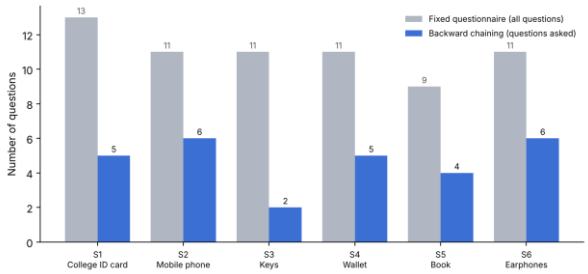


Fig. 3: Questions asked by backward chaining versus a fixed questionnaire

C. Advantages, Limitations, and Applications

Advantages: systematic rather than random search, fewer questions, explainable results, handling of uncertainty through certainty factors and "Not sure" answers, and easy extension by adding data to the knowledge base.

Limitations: scenarios use scripted answers consistent with the expected location, so they test the correctness of the reasoning rather than real-world accuracy; certainty factors were chosen by the developers and not calibrated on real data; the knowledge base is small; and evidence from several rules pointing to the same location is not combined. No user-study accuracy is reported.

Applications: helping students and others locate everyday belongings, teaching knowledge representation and backward chaining, and serving as a base for assistants that reason about household or campus objects.

VI. Discussion

A. Why Backward Chaining

Information for this task comes from the user, and each question costs time and effort. Starting from the goal and seeking only the facts relevant to the current hypothesis keeps questioning short. Trying rules in order of certainty factor tests the most reliable explanations first and stops as soon as one is confirmed. Static facts also help: for a non-chargeable item such as a wallet, Chargeable(?i) fails at once and the charging question is never asked.

B. Explainability

Because every conclusion comes from explicit facts and rules, the reasoning trace lets users see why a location was suggested and judge whether it is plausible. Confidence is stated as a percentage, and low-confidence fallbacks are clearly labelled rather than overstated.

C. Practical Considerations

The quality of the output depends on the user's answers and on the rules and certainty factors in the knowledge base. Since the certainty factors are developer-chosen and the evaluation uses scripted scenarios, the system should be read as a reasoning aid, not as a guarantee of finding the item.

VII. Conclusion and Future Work

A. Conclusion

This paper presented the AI Lost Item Finder, which applies Knowledge Representation, First-Order Logic, and backward chaining to locating misplaced belongings. Knowledge about items, containers, activities, and locations is stored as explicit facts and rules, and a backward-chaining engine with unification works from location

hypotheses back to supporting facts, asking only what is needed. In six scripted scenarios covering every rule, the system returned the expected location each time, asked about 58% fewer questions than a fixed questionnaire, and explained every conclusion with a complete trace. These results do not measure real-world accuracy.

B. Future Work

Future work includes a user study measuring top-1 and top-3 accuracy against where items were actually found, with comparison to a fixed questionnaire and unaided searching; combining certainty factors when several rules support one location; learning rule confidences from past sessions; time-based reasoning about the order of activities; a larger knowledge base; a graphical or mobile interface; optional tracking data such as the last Bluetooth connection of earphones; and voice input. These are development directions, not results established by this work.

VIII. References

[1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2020.

[2] B. G. Buchanan and E. H. Shortliffe, Rule-Based Expert Systems: The MYCIN Experiments of the Stanford Heuristic Programming Project. Addison-Wesley, 1984.

[3] E. H. Shortliffe and B. G. Buchanan, "A model of inexact reasoning in medicine," Mathematical Biosciences, vol. 23, no. 3–4, pp. 351–379, 1975.

[4] J. C. Giarratano and G. D. Riley, Expert Systems: Principles and Programming, 4th ed. Thomson Course Technology, 2005.

[5] P. Jackson, Introduction to Expert Systems, 3rd ed. Addison-Wesley, 1998.

[6] R. Kowalski, "Predicate logic as programming language," in Proc. IFIP Congress 74, 1974, pp. 569–574.

[7] I. Bratko, Prolog Programming for Artificial Intelligence, 4th ed. Pearson, 2012.

[8] Python Software Foundation, Python 3 Documentation. [Online]. Available: https://docs.python.org/3/